

Classificador de Grãos de Soja

Relatório Técnico — Projeto Integrador FATEC

Gustavo Barros · 2026

1. Visão Geral do Projeto

Sistema de classificação visual automatizada de grãos de soja por imagem. O usuário fotografa grãos com o celular; o sistema segmenta cada grão individualmente, classifica em uma das 5 classes de qualidade e exibe bounding boxes anotados na tela.

O fluxo completo é: **foto do celular** → **segmentação OpenCV** → **EfficientNet-B0** → **bounding boxes + rótulos na tela**. A interface roda no navegador via Gradio, hospedada gratuitamente no Hugging Face Spaces.

Classes de qualidade (5)

Índice	Nome (EN)	Rótulo PT	Descrição
0	Broken soybeans	Quebrado	Grão partido ou com fragmento faltando
1	Immature soybeans	Imaturo	Verde ou desenvolvimento incompleto
2	Intact soybeans	Intacto	Grão saudável, sem defeitos
3	Skin-damaged soybeans	Casca danificada	Casca rasgada, descascada ou abrasada
4	Spotted soybeans	Manchado	Manchas escuras ou colorações anormais

2. Stack Tecnológico

Camada	Tecnologia	Versão	Papel no projeto
Framework ML	TensorFlow / Keras	≥ 2.15	Treino, inferência e fine-tuning do modelo
Modelo base	EfficientNet-B0	ImageNet	Backbone pré-treinado para transfer learning
Segmentação	OpenCV	≥ 4.9	Recorte de cada grão da foto (threshold + contornos)
Interface	Gradio	≥ 4.44	App web/mobile sem código front-end
Deploy	Hugging Face Spaces	—	Hospedagem gratuita, link compartilhável
Dataset HF	HF Datasets	—	Armazenamento das correções manuais

Treino (GPU)	Google Colab	T4 GPU	Treinamento gratuito na nuvem
Versionamento	Git + Git LFS	—	Modelo .keras (29 MB) via Large File Storage
Dependências	Pillow / NumPy	—	Manipulação de imagens e arrays

3. Dataset de Treinamento

Atributo	Valor
Nome	SoyaBeans Classifications v2
Fonte	Roboflow Universe — hansaka-sudusinghe
Licença	MIT (uso comercial liberado)
Total de imagens	12.528 (com augmentation Roboflow)
Dimensão original	400 × 400 px
Dimensão no modelo	224 × 224 px (resize no pipeline Keras)
Splits	train / valid / test (pré-definidos)
Treino	10.905 imagens
Validação	1.038 imagens
Teste	527 imagens
Augmentations Roboflow	Flip horizontal/vertical, rotação, crop, shear
Obs.	Pasta 'Part of the original soybean images' excluída via class_names explícito

4. Arquitetura do Modelo

4.1 Transfer Learning — EfficientNet-B0

A EfficientNet-B0 é uma CNN composta por blocos MBConv (Mobile Inverted Residual) com squeeze-and-excitation. Pré-treinada no ImageNet (1,2 M imagens, 1000 classes), ela extrai features visuais ricas. O projeto substitui apenas a cabeça de classificação por uma nova camada densa de 5 saídas (softmax), mantendo o backbone intacto na Fase 1.

Componente	Detalhe
Backbone	EfficientNetB0 (ImageNet, include_top=False)
Input shape	(224, 224, 3)
Pré-processamento	efficientnet.preprocess_input (normalização específica da rede)
Pooling	GlobalAveragePooling2D

Regularização	BatchNormalization + Dropout(0.3)
Cabeça	Dense(5, activation='softmax')
Parâmetros treináveis (Fase 1)	~8.965 (só a cabeça)
Parâmetros congelados (Fase 1)	~4.052.151 (backbone)
Parâmetros treináveis (Fase 2)	~últimas 30 camadas descongeladas

4.2 Estratégia de Treino em 2 Fases

	Fase 1 — Cabeça	Fase 2 — Fine-tuning
Base	Congelada (trainable=False)	Últimas 30 camadas descongeladas
BatchNorm	—	Mantida congelada (preserva estatísticas ImageNet)
LR	1e-3 (Adam)	1e-5 (Adam) — 100× menor
Épocas	até 25 (EarlyStopping)	até 15 (EarlyStopping)
Monitor	val_accuracy	val_accuracy
Paciência	5 épocas	5 épocas
ReduceLR	patience=3, factor=0.5	patience=3, factor=0.5

4.3 Balanceamento de Classes

O dataset tem distribuição desigual entre classes (spotted tem menos imagens). Para compensar, usa-se `class_weight` calculado via `sklearn.utils.class_weight.compute_class_weight('balanced')`, que aumenta o peso das classes minoritárias na função de perda, evitando que o modelo ignore-as.

4.4 Augmentation de Dados

Técnica	Parâmetro	Aplicada em
RandomFlip	horizontal_and_vertical	Treino
RandomRotation	fator=0.1	Treino
RandomZoom	fator=0.08	Treino
RandomBrightness	max_delta=0.1	Treino
RandomContrast	fator=0.1	Treino

5. Pipeline de Inferência (app.py)

5.1 Modo Um Grão (aba 'Um grão')

Recebe a foto, detecta o maior objeto via segmentação OpenCV e retorna a classe com as probabilidades por classe.

Etapas: **(1)** Grayscale → GaussianBlur(7×7) → Threshold Otsu → Morfologia (CLOSE + OPEN com kernel elíptico 7×7) → findContours → maior contorno → boundingRect + padding → crop. **(2)** Crop redimensionado para 224×224 → EfficientNet → softmax → classe + confiança.

5.2 Modo Vários Grãos (aba 'Vários grãos')

Detecta todos os contornos válidos da imagem (filtro por área mínima/máxima relativa ao frame), classifica cada recorte individualmente e desenha bounding boxes coloridos por classe sobre a imagem original. Grãos com confiança abaixo do limiar ajustável (padrão 60%) são marcados com '?'.

5.3 Segmentação OpenCV — Parâmetros

Passo	Método	Parâmetro
Suavização	GaussianBlur	kernel 7×7
Binarização	threshold Otsu	THRESH_BINARY + THRESH_OTSU
Fechamento de buracos	morphologyEx CLOSE	kernel elíptico 7×7, 2 iterações
Remoção de ruído	morphologyEx OPEN	kernel elíptico 7×7, 1 iteração
Deteção	findContours	RETR_EXTERNAL, CHAIN_APPROX_SIMPLE
Filtro de área	cv2.contourArea	0,15% – 25% da área total do frame
Padding	max(15, 12% do menor lado)	—

6. Deploy — Hugging Face Spaces

Item	Detalhe
Space	Guguinhaxd/soja-inspection (SDK: Gradio)
Arquivo principal	app.py (raiz do repositório do Space)
Modelo	soja_model_final.keras — 29 MB via Git LFS
Classes	soja_classes.json
Dependências	requirements.txt (tensorflow, gradio, opencv-python-headless, Pillow, numpy, huggingface_hub)
Hardware	CPU Free Tier (sem GPU em produção)
Secret configurado	SOJA_CORRECTIONS — token HF com permissão de escrita no dataset

O modelo .keras (29 MB) excede o limite de 10 MB do Git padrão e por isso requer **Git LFS (Large File Storage)**: `git lfs install` → `git lfs track '*.keras'` → `commit` → `push`.

7. Feedback Loop — Active Learning

Quando o modelo classifica um grão incorretamente, o usuário seleciona a classe correta no app e clica em 'Enviar correção'. A imagem é salva automaticamente no dataset **Guguinhaxd/soja-correction** via `HfApi.upload_file()`, organizadas em subpastas por classe.

Etapa	Descrição
1. Coleta	Usuário corrige classificação errada no app → imagem salva no HF dataset
2. Acúmulo	Dataset cresce com exemplos reais do setup do usuário (resolução do domain shift)
3. Fine-tuning	Notebook finetune.ipynb baixa correções, mistura com dataset original e re-treina
4. Deploy	Modelo melhorado enviado ao Space via <code>HfApi.upload_file()</code> (sem git manual)

Por que o grão é salvo já recortado:

O modelo classifica o crop (grão preenchendo o quadro), não a foto inteira. Salvar a imagem já recortada garante que treino e inferência usem o mesmo enquadramento — evita domain shift interno.

8. Fine-tuning com Dados Reais (finetune.ipynb)

8.1 Problema: Domain Shift

O modelo foi treinado com fotos de laboratório (dataset Roboflow). Fotos tiradas com celular em condições domésticas parecem visualmente diferentes — isso causa queda de acurácia na prática. O fine-tuning com fotos reais corrige isso diretamente.

8.2 Estratégia — Pesos das Correções

Parâmetro	Valor	Razão
CORR_WEIGHT	2.0 / 1.5	Cada correção conta 2× na loss (gradiente dobrado). Reduz para 1.5 automaticamente quando o modelo estiver mais seguro.
Mix de dados	70% / 30%	70% das amostras de cada lote vêm das correções, 30% do dataset original (anti-catastrophic forgetting)
Influência total	~4,7×	70% presença × 2.0 de peso = 4,7× mais influência que um exemplo do dataset original

8.3 Anti-Catastrophic Forgetting

Catastrophic forgetting ocorre quando o modelo é treinado apenas com novos dados e 'esquece' o que aprendeu antes. A solução usada aqui é o **Experience Replay**: 30% de cada lote de treino vem do dataset original, garantindo que o modelo continue acertando as imagens de laboratório enquanto aprende as do celular.

8.4 Augmentation por Fonte

Fonte	Augmentation	Parâmetros
-------	--------------	------------

Correções (reais)	Forte (aug_strong)	Flip H+V, rotação 25%, zoom 15%, translação 12%, brilho 25%, contraste 25%
Dataset original	Leve (aug_light)	Flip H, rotação 5%, brilho 5%

8.5 Hiperparâmetros do Fine-tuning

Parâmetro	Valor
Optimizer	Adam
Learning rate	1e-5 (100× menor que o treino original)
Camadas descongeladas	Últimas 30 da base EfficientNet (BatchNorm mantida congelada)
Epochs	até 20 (EarlyStopping patience=6, monitor=val_accuracy)
ReduceLROnPlateau	patience=3, factor=0.5, min_lr=1e-7
sample_weight	Embutido no tf.data.Dataset — Keras aplica automaticamente no fit()
Validação	20% das correções separadas (mede acurácia no domínio real)
Trava de segurança	Só sobrescreve soja_model_final.keras se val melhorou E teste original ≥ 70%

9. Resultados Observados

Rodada	Correções	Acurácia (fotos reais)	Observação
Modelo original	0	~29%	Domain shift severo — confundia tudo com Quebrado 9
Fine-tuning #1	57	64%	Overfitting: treino 89% / val 64% — dados insuficientes
Fine-tuning #2 (planejado)	600	~92–96% (estimado)	Volume suficiente para generalização robusta

Análise do overfitting na Rodada 1:

Com apenas 57 fotos e CORR_WEIGHT=2.0, o modelo memorizou os exemplos de treino (89%) mas não generalizou para a validação (64%). O gap de 25 pontos percentuais indica dados insuficientes para o nível de peso aplicado. Com ~600 fotos, o modelo é forçado a aprender padrões reais em vez de memorizar, reduzindo o gap para <10pp.

10. Estrutura de Arquivos do Projeto

Arquivo	Localização	Função
train.ipynb	model/	Notebook Colab: treino completo EfficientNet-B0 em 2 fases
finetune.ipynb	model/	Notebook Colab: fine-tuning com correções reais do HF dataset
app.py	model/ (Space: raiz)	Gradio app: inferência + feedback loop
requirements.txt	model/ (Space: raiz)	Dependências Python para o HF Space

soja_classes.json	model/ (Space: raiz)	Lista de classes na ordem correta (índice = saída do modelo)
soja_model_final.keras	Space (raiz)	Modelo treinado — 29 MB, versionado via Git LFS
CLAUDE.md	raiz do repo	Contexto técnico do projeto para desenvolvimento assistido

11. Glossário Técnico

Conceito	Definição aplicada ao projeto
Transfer Learning	Reutiliza pesos ImageNet da EfficientNet-B0; treina só a cabeça de 5 classes
Fine-tuning	Descongela as últimas 30 camadas do backbone com LR baixo (1e-5)
BatchNormalization	Mantida congelada no fine-tuning para preservar estatísticas do ImageNet
Domain Shift	Diferença entre as fotos do dataset (lab) e as fotos do celular do usuário
Active Learning	Loop onde o modelo aprende continuamente com correções feitas pelo usuário
Catastrophic Forgetting	Fenômeno em que o modelo 'esquece' o que aprendeu ao treinar com novos dados
Experience Replay	Solução: misturar 30% de dados antigos no treino de fine-tuning
sample_weight	Peso por exemplo na loss function — correções têm peso 2× (gradiente dobrado)
class_weight	Peso por classe — compensa desbalanceamento (computed via sklearn)
Otsu Threshold	Binarização automática que encontra o threshold ótimo por análise de histograma
Git LFS	Large File Storage — necessário para versionar o .keras de 29 MB no HF
EarlyStopping	Para o treino quando val_accuracy não melhora por N épocas consecutivas
ReduceLROnPlateau	Reduz o LR pela metade quando val_loss para de cair